

02 — Smoke test: FedSafe-Fuse and IFCNN on a small MedMNIST subset

Phase 2 of FedSafe-Fuse. Runs on Google Colab Free.

What this notebook does

1. Mounts Drive and re-uses the paired MedMNIST arrays from Phase 1.
2. Defines the FedSafe-Fuse model (~5M params) and the IFCNN baseline.
3. Trains both for 5 epochs on a 256-image MedMNIST subset.
4. Reports loss / SSIM / PSNR on a held-out 32-sample subset.
5. Renders a qualitative panel (MRI, PET, fused outputs).
6. **Measures seconds-per-epoch** and extrapolates to the full federated schedule (K=3 clients × T=50 rounds × E=5 local epochs) so we can decide whether the proposal config fits Colab Free session limits.

Outputs

- `MyDrive/FedSafeFuse/results/smoke_test.csv` — loss/SSIM/PSNR per epoch
- `MyDrive/FedSafeFuse/figures/smoke_qualitative.png` — 2-row qualitative panel
- `MyDrive/FedSafeFuse/results/timing_extrapolation.txt` — full schedule estimate

Seed = 42 throughout.

```
# 0. Install deps + seeds
import os, sys, time, json, random, gc
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print('Device:', DEVICE)
print('Torch:', torch.__version__)
```

```
Device: cuda
Torch: 2.11.0+cu128
```

```
# 1. Mount Drive (skip if already mounted)
if not os.path.exists('/content/drive/MyDrive'):
    from google.colab import drive
    drive.mount('/content/drive')
PROJECT_DRIVE = '/content/drive/MyDrive/FedSafeFuse'
os.makedirs(f'{PROJECT_DRIVE}/results', exist_ok=True)
os.makedirs(f'{PROJECT_DRIVE}/figures', exist_ok=True)
print('Project root:', PROJECT_DRIVE)
```

```
Mounted at /content/drive
Project root: /content/drive/MyDrive/FedSafeFuse
```

Model and loss definitions

The four cells below are extracted verbatim from `src/models/fedsafe.py`, `src/models/ifcnn.py`, `src/losses.py`, `src/data.py`. They live inline so the notebook is fully self-contained — no Drive code upload required.

```
# FedSafe-Fuse model
"""FedSafe-Fuse: dual MobileNetV3-Small encoders + 2-layer Transformer + conv decoder.

Per the project proposal:
- Two modality-specific encoders, each MobileNetV3-Small (~2.5M params total each)
- Shared 2-layer Transformer module (2 heads, embed dim 128) for cross-modal attention
- Convolutional decoder
- Target ~8M params

Actual realised parameter count depends on decoder width; default config lands at ~5M.
```

```

"""

from __future__ import annotations

import torch
import torch.nn as nn
from torchvision.models import mobilenet_v3_small

class ModalityEncoder(nn.Module):
    """MobileNetV3-Small features adapted to 1-channel input.

    MobileNetV3-Small downsamples by 32x, so a 128x128 input yields 4x4 features.
    We bilinearly upsample to 8x8 to give the cross-modal Transformer 64 tokens
    per modality (matching the proposal). The upsample is parameter-free.
    """

    def __init__(self, embed_dim: int = 128):
        super().__init__()
        base = mobilenet_v3_small(weights=None)
        # Replace first conv to accept a single channel (medical grayscale)
        base.features[0][0] = nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1, bias=False)
        self.features = base.features # (B, 576, 4, 4) for 128x128 input
        self.proj = nn.Conv2d(576, embed_dim, kernel_size=1)
        self.upsample = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.features(x)
        h = self.proj(h)
        return self.upsample(h) # (B, embed_dim, 8, 8)

class CrossModalTransformer(nn.Module):
    """2-layer Transformer encoder over concatenated MRI/PET tokens.

    Input feature maps are flattened to 64 spatial tokens per modality, given a
    learned modality embedding, concatenated to 128 tokens, given a learned
    positional embedding, and passed through `num_layers` transformer encoder layers.
    Output is averaged across the two modality halves and reshaped back to a
    spatial feature map for the decoder.
    """

    def __init__(
        self,
        embed_dim: int = 128,
        num_layers: int = 2,
        num_heads: int = 2,
        ff_dim: int = 256,
        dropout: float = 0.1,
        spatial_tokens_per_modality: int = 64,
    ):
        super().__init__()
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=ff_dim,
            dropout=dropout,
            batch_first=True,
            activation="gelu",
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
        total_tokens = spatial_tokens_per_modality * 2
        self.pos_embed = nn.Parameter(torch.zeros(1, total_tokens, embed_dim))
        nn.init.trunc_normal_(self.pos_embed, std=0.02)
        self.mod_embed = nn.Embedding(2, embed_dim) # 0 = MRI, 1 = PET
        self.spatial_tokens = spatial_tokens_per_modality

    def forward(self, mri_feat: torch.Tensor, pet_feat: torch.Tensor) -> torch.Tensor:
        # mri_feat, pet_feat: (B, C, H, W) with H*W = self.spatial_tokens
        B, C, H, W = mri_feat.shape
        mri_tokens = mri_feat.flatten(2).transpose(1, 2) # (B, HW, C)
        pet_tokens = pet_feat.flatten(2).transpose(1, 2)
        mri_tokens = mri_tokens + self.mod_embed.weight[0]
        pet_tokens = pet_tokens + self.mod_embed.weight[1]
        tokens = torch.cat([mri_tokens, pet_tokens], dim=1) + self.pos_embed
        out = self.transformer(tokens)
        mri_out = out[:, : self.spatial_tokens].transpose(1, 2).reshape(B, C, H, W)
        pet_out = out[:, self.spatial_tokens :].transpose(1, 2).reshape(B, C, H, W)
        return 0.5 * (mri_out + pet_out)

class FusionDecoder(nn.Module):

```

```

"""Progressive bilinear upsample + conv decoder: 8x8 -> 16 -> 32 -> 64 -> 128."""

def __init__(self, embed_dim: int = 128, base_ch: int = 256):
    super().__init__()
    c0, c1, c2, c3 = base_ch, base_ch, base_ch // 2, base_ch // 4
    self.up1 = self._block(embed_dim, c0)
    self.up2 = self._block(c0, c1)
    self.up3 = self._block(c1, c2)
    self.up4 = self._block(c2, c3)
    self.final = nn.Conv2d(c3, 1, kernel_size=1)

    @staticmethod
    def _block(in_ch: int, out_ch: int) -> nn.Sequential:
        return nn.Sequential(
            nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False),
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.GELU(),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.GELU(),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.up1(x)
        h = self.up2(h)
        h = self.up3(h)
        h = self.up4(h)
        return torch.sigmoid(self.final(h))

class FedSafeFuse(nn.Module):
    """End-to-end FedSafe-Fuse model. Inputs: two (B, 1, 128, 128) tensors. Output: (B, 1, 128, 128)."""

    def __init__(
        self,
        embed_dim: int = 128,
        transformer_layers: int = 2,
        transformer_heads: int = 2,
        transformer_ff: int = 256,
        decoder_base_ch: int = 256,
    ):
        super().__init__()
        self.mri_encoder = ModalityEncoder(embed_dim)
        self.pet_encoder = ModalityEncoder(embed_dim)
        self.fusion = CrossModalTransformer(
            embed_dim=embed_dim,
            num_layers=transformer_layers,
            num_heads=transformer_heads,
            ff_dim=transformer_ff,
        )
        self.decoder = FusionDecoder(embed_dim=embed_dim, base_ch=decoder_base_ch)

    def forward(self, mri: torch.Tensor, pet: torch.Tensor) -> torch.Tensor:
        mri_feat = self.mri_encoder(mri)
        pet_feat = self.pet_encoder(pet)
        fused_feat = self.fusion(mri_feat, pet_feat)
        return self.decoder(fused_feat)

def count_parameters(model: nn.Module) -> int:
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

```

```

# IFCNN baseline
"""IFCNN baseline (Zhang et al. 2021).

A general image fusion framework based on a two-stream CNN with element-wise fusion
followed by reconstruction. Simplified from the original paper for our 128x128
single-channel fusion setting. Used as Baseline 2 in the FedSafe-Fuse evaluation.
"""

from __future__ import annotations

import torch
import torch.nn as nn

class IFCNN(nn.Module):
    """Two-stream conv + element-wise fusion + conv reconstruction."""

```

```

FUSION_MODES = ("max", "mean", "sum")

def __init__(self, base_ch: int = 64, fusion: str = "max"):
    super().__init__()
    assert fusion in self.FUSION_MODES, f"fusion must be one of {self.FUSION_MODES}"
    self.fusion_mode = fusion
    self.stream1 = self._make_stream(base_ch)
    self.stream2 = self._make_stream(base_ch)
    self.recon = nn.Sequential(
        nn.Conv2d(base_ch, base_ch, 3, padding=1),
        nn.BatchNorm2d(base_ch),
        nn.ReLU(inplace=True),
        nn.Conv2d(base_ch, base_ch, 3, padding=1),
        nn.BatchNorm2d(base_ch),
        nn.ReLU(inplace=True),
        nn.Conv2d(base_ch, base_ch // 2, 3, padding=1),
        nn.BatchNorm2d(base_ch // 2),
        nn.ReLU(inplace=True),
        nn.Conv2d(base_ch // 2, 1, kernel_size=1),
    )

    @staticmethod
    def _make_stream(c: int) -> nn.Sequential:
        return nn.Sequential(
            nn.Conv2d(1, c, 3, padding=1),
            nn.BatchNorm2d(c),
            nn.ReLU(inplace=True),
            nn.Conv2d(c, c, 3, padding=1),
            nn.BatchNorm2d(c),
            nn.ReLU(inplace=True),
            nn.Conv2d(c, c, 3, padding=1),
            nn.BatchNorm2d(c),
            nn.ReLU(inplace=True),
            nn.Conv2d(c, c, 3, padding=1),
            nn.BatchNorm2d(c),
            nn.ReLU(inplace=True),
        )

    def forward(self, x1: torch.Tensor, x2: torch.Tensor) -> torch.Tensor:
        f1 = self.stream1(x1)
        f2 = self.stream2(x2)
        if self.fusion_mode == "max":
            fused = torch.maximum(f1, f2)
        elif self.fusion_mode == "mean":
            fused = 0.5 * (f1 + f2)
        else:
            fused = f1 + f2
        return torch.sigmoid(self.recon(fused))

def count_parameters(model: nn.Module) -> int:
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

```



```

# Composite fusion loss + eval metrics
"""Composite fusion loss: L1 + beta * (1 - SSIM).

Implements a differentiable SSIM via a Gaussian-windowed convolution
(no external dependency on pytorch_msssim). Designed for single-channel
medical images in [0, 1].
"""

from __future__ import annotations

import torch
import torch.nn as nn
import torch.nn.functional as F

def _gaussian_window(window_size: int, sigma: float, channels: int) -> torch.Tensor:
    coords = torch.arange(window_size, dtype=torch.float32) - (window_size - 1) / 2
    g = torch.exp(-(coords ** 2) / (2 * sigma ** 2))
    g = g / g.sum()
    w_2d = g.unsqueeze(0) * g.unsqueeze(1) # (W, W)
    return w_2d.expand(channels, 1, window_size, window_size).contiguous()

class SSIMLoss(nn.Module):
    """1 - SSIM, computed with a Gaussian sliding window. Single-channel input in [0, 1]."""

    def __init__(

```

```

        self,
        window_size: int = 11,
        sigma: float = 1.5,
        data_range: float = 1.0,
        channels: int = 1,
    ):
        super().__init__()
        self.window_size = window_size
        self.data_range = data_range
        self.C1 = (0.01 * data_range) ** 2
        self.C2 = (0.03 * data_range) ** 2
        self.register_buffer("window", _gaussian_window(window_size, sigma, channels))

    def _filter(self, x: torch.Tensor) -> torch.Tensor:
        return F.conv2d(x, self.window, padding=self.window_size // 2, groups=x.shape[1])

    def forward(self, pred: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
        mu_p = self._filter(pred)
        mu_t = self._filter(target)
        mu_p2 = mu_p ** 2
        mu_t2 = mu_t ** 2
        mu_pt = mu_p * mu_t
        sigma_p2 = self._filter(pred * pred) - mu_p2
        sigma_t2 = self._filter(target * target) - mu_t2
        sigma_pt = self._filter(pred * target) - mu_pt
        num = (2 * mu_pt + self.C1) * (2 * sigma_pt + self.C2)
        den = (mu_p2 + mu_t2 + self.C1) * (sigma_p2 + sigma_t2 + self.C2)
        ssim_map = num / den
        return 1.0 - ssim_map.mean()

class FusionLoss(nn.Module):
    """L1(fused, ref) + beta * SSIMLoss(fused, ref).

    Reference composite is a per-pixel weighted average of source modalities, per the proposal.
    Default weights are (0.5, 0.5). Override `reference_weights` per dataset if needed.
    """

    def __init__(self, beta: float = 1.0, reference_weights: tuple[float, float] = (0.5, 0.5)):
        super().__init__()
        assert abs(sum(reference_weights) - 1.0) < 1e-6, "reference_weights must sum to 1.0"
        self.l1 = nn.L1Loss()
        self.ssim = SSIMLoss()
        self.beta = beta
        w1, w2 = reference_weights
        self.register_buffer("w1", torch.tensor(w1))
        self.register_buffer("w2", torch.tensor(w2))

    def reference(self, src1: torch.Tensor, src2: torch.Tensor) -> torch.Tensor:
        return self.w1 * src1 + self.w2 * src2

    def forward(
        self,
        fused: torch.Tensor,
        src1: torch.Tensor,
        src2: torch.Tensor,
    ) -> torch.Tensor:
        ref = self.reference(src1, src2)
        return self.l1(fused, ref) + self.beta * self.ssim(fused, ref)

@torch.no_grad()
def ssim_score(pred: torch.Tensor, target: torch.Tensor) -> float:
    """SSIM (higher is better) for evaluation. Mirrors SSIMLoss without the 1 - ... wrap."""
    loss_fn = SSIMLoss().to(pred.device)
    return float(1.0 - loss_fn(pred, target).item())

@torch.no_grad()
def psnr_score(pred: torch.Tensor, target: torch.Tensor, data_range: float = 1.0) -> float:
    """PSNR in dB."""
    mse = F.mse_loss(pred, target).item()
    if mse == 0:
        return float("inf")
    return 10.0 * torch.log10(torch.tensor(data_range ** 2 / mse)).item()

```

```

# Dataset wrappers
"""PyTorch Datasets for FedSafe-Fuse: MedMNIST paired arrays and BraTS per-case slices.

Designed to read directly from Google Drive (`/content/drive/MyDrive/FedSafeFuse/`)
in Colab, or from a local copy of that tree.

```

```

"""

from __future__ import annotations

import json
import os
from collections import OrderedDict
from typing import Iterable

import numpy as np
import torch
from torch.utils.data import Dataset

class MedMNISTPaired(Dataset):
    """Pre-computed (MRI, PET, label) triples from a single .npz on Drive.

    Optional `indices` restricts to a subset of the global array (used for per-client
    partitions and calibration hold-outs).
    """

    def __init__(self, npz_path: str, indices: Iterable[int] | None = None):
        with np.load(npz_path) as d:
            self.mri = d["mri"] # (N, 128, 128) float32 in [0, 1]
            self.pet = d["pet"]
            self.labels = d["labels"].astype(np.int64).flatten()
            self.indices = list(indices) if indices is not None else list(range(len(self.mri)))

    def __len__(self) -> int:
        return len(self.indices)

    def __getitem__(self, i: int):
        gi = self.indices[i]
        mri = torch.from_numpy(self.mri[gi]).unsqueeze(0) # (1, 128, 128)
        pet = torch.from_numpy(self.pet[gi]).unsqueeze(0)
        return mri, pet, int(self.labels[gi])

class BraTSPaired(Dataset):
    """Lazily loads BraTS per-case .npz files, indexed by (case_id, slice_idx).

    A small LRU cache keeps the most recently used cases in memory to avoid
    repeated Drive reads when iterating sequentially.
    """

    def __init__(self, brats_out_dir: str, slice_list, cache_size: int = 8):
        self.brats_out_dir = brats_out_dir
        self.slice_list = list(slice_list) # list of [case_id, slice_idx]
        self.cache_size = cache_size
        self._cache: "OrderedDict[str, tuple[np.ndarray, np.ndarray]]" = OrderedDict()

    def __len__(self) -> int:
        return len(self.slice_list)

    def _load(self, case_id: str):
        if case_id in self._cache:
            self._cache.move_to_end(case_id)
            return self._cache[case_id]
        path = os.path.join(self.brats_out_dir, f"{case_id}.npz")
        d = np.load(path)
        t1, t2 = d["t1"], d["t2"]
        d.close()
        self._cache[case_id] = (t1, t2)
        if len(self._cache) > self.cache_size:
            self._cache.popitem(last=False)
        return t1, t2

    def __getitem__(self, i: int):
        case_id, sidx = self.slice_list[i]
        t1, t2 = self._load(case_id)
        return (
            torch.from_numpy(t1[sidx]).unsqueeze(0),
            torch.from_numpy(t2[sidx]).unsqueeze(0),
            case_id,
        )

    def load_partition(json_path: str) -> dict:
        """Read a partition manifest JSON saved by Phase 1."""
        with open(json_path) as f:
            return json.load(f)

```

```
# 2. Build models, report parameter counts
fs = FedSafeFuse().to(DEVICE)
ic = IFCNN().to(DEVICE)
n_fs = count_parameters(fs)
n_ic = sum(p.numel() for p in ic.parameters() if p.requires_grad)
print(f'FedSafe-Fuse: {n_fs:,} params ({n_fs/1e6:.2f}M)')
print(f'IFCNN baseline: {n_ic:,} params ({n_ic/1e6:.2f}M)')
```

```
FedSafe-Fuse: 4,904,385 params (4.90M)
IFCNN baseline: 316,545 params (0.32M)
```

```
# 3. Build a 256-sample MedMNIST subset and a 32-sample held-out subset
with open(f'{PROJECT_DRIVE}/partitions/medmnist_K3.json') as f:
    medmnist_partition = json.load(f)

# Take the first 256 indices from Client 0 for training, last 32 from Client 0 for eval
client0_idx = medmnist_partition['client_indices'][0]
train_idx = client0_idx[:256]
eval_idx = client0_idx[-32:]
print(f'Train subset: {len(train_idx)} | Eval subset: {len(eval_idx)}')

train_ds = MedMNISTPaired(f'{PROJECT_DRIVE}/medmnist/train_paired.npz', indices=train_idx)
eval_ds = MedMNISTPaired(f'{PROJECT_DRIVE}/medmnist/train_paired.npz', indices=eval_idx)

train_loader = DataLoader(train_ds, batch_size=16, shuffle=True, num_workers=0)
eval_loader = DataLoader(eval_ds, batch_size=16, shuffle=False, num_workers=0)

# Verify one batch
mri, pet, lbl = next(iter(train_loader))
print(f'Batch: mri={tuple(mri.shape)} pet={tuple(pet.shape)} range=[{mri.min():.2f},{mri.max():.2f}]')
```

```
Train subset: 256 | Eval subset: 32
Batch: mri=(16, 1, 128, 128) pet=(16, 1, 128, 128) range=[0.00,1.00]
```

```
# 4. Smoke-train one model for N epochs, return per-epoch metrics + timing
@torch.no_grad()
def evaluate(model, loader, loss_fn):
    model.eval()
    ssims, psnrs, losses = [], [], []
    for mri, pet, _ in loader:
        mri = mri.to(DEVICE); pet = pet.to(DEVICE)
        fused = model(mri, pet)
        ref = 0.5 * (mri + pet)
        losses.append(loss_fn(fused, mri, pet).item())
        ssims.append(ssim_score(fused, ref))
        psnrs.append(psnr_score(fused, ref))
    return float(np.mean(losses)), float(np.mean(ssims)), float(np.mean(psnrs))

def smoke_train(model, name, epochs=5, lr=1e-3, beta=1.0):
    loss_fn = FusionLoss(beta=beta).to(DEVICE)
    opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
    history = []
    print(f'\n=== Training {name} for {epochs} epochs ===')
    for epoch in range(1, epochs + 1):
        model.train()
        t0 = time.time()
        epoch_losses = []
        n_batches = 0
        for mri, pet, _ in train_loader:
            mri = mri.to(DEVICE); pet = pet.to(DEVICE)
            fused = model(mri, pet)
            loss = loss_fn(fused, mri, pet)
            opt.zero_grad(); loss.backward(); opt.step()
            epoch_losses.append(loss.item())
            n_batches += 1
        train_loss = float(np.mean(epoch_losses))
        epoch_secs = time.time() - t0
        eval_loss, eval_ssim, eval_psnr = evaluate(model, eval_loader, loss_fn)
        history.append({
            'epoch': epoch, 'name': name,
            'train_loss': train_loss, 'eval_loss': eval_loss,
            'eval_ssim': eval_ssim, 'eval_psnr': eval_psnr,
            'epoch_secs': epoch_secs, 'batches_per_epoch': n_batches,
        })
        print(f' ep{epoch} | loss={train_loss:.4f} | eval_loss={eval_loss:.4f} | '
              f'SSIM={eval_ssim:.4f} | PSNR={eval_psnr:.2f}dB | {epoch_secs:.1f}s/epoch')
    return history

history_fs = smoke_train(fs, 'FedSafe-Fuse')
```

```
history_ic = smoke_train(ic, 'IFCNN')
```

```
=== Training FedSafe-Fuse for 5 epochs ===
```

ep1	loss=1.0423	eval_loss=1.0740	SSIM=0.1851	PSNR=10.42dB	4.5s/epoch
ep2	loss=0.9161	eval_loss=1.1223	SSIM=0.1879	PSNR=7.85dB	2.3s/epoch
ep3	loss=0.8531	eval_loss=1.0971	SSIM=0.2029	PSNR=8.14dB	2.4s/epoch
ep4	loss=0.8108	eval_loss=1.0731	SSIM=0.1817	PSNR=10.56dB	2.4s/epoch
ep5	loss=0.7887	eval_loss=1.0734	SSIM=0.1819	PSNR=10.53dB	2.5s/epoch

```
=== Training IFCNN for 5 epochs ===
```

ep1	loss=0.4193	eval_loss=0.5899	SSIM=0.5487	PSNR=15.92dB	2.7s/epoch
ep2	loss=0.2589	eval_loss=0.4028	SSIM=0.6888	PSNR=19.31dB	2.6s/epoch
ep3	loss=0.1832	eval_loss=0.2985	SSIM=0.7573	PSNR=23.82dB	2.6s/epoch
ep4	loss=0.1093	eval_loss=0.0733	SSIM=0.9502	PSNR=29.65dB	2.7s/epoch
ep5	loss=0.0619	eval_loss=0.0262	SSIM=0.9835	PSNR=36.20dB	2.6s/epoch

```
# 5. Save per-epoch metrics CSV
```

```
import csv
```

```
csv_path = f'{PROJECT_DRIVE}/results/smoke_test.csv'
```

```
with open(csv_path, 'w', newline='') as f:
```

```
    w = csv.DictWriter(f, fieldnames=list(history_fs[0].keys()))
```

```
    w.writeheader()
```

```
    for row in history_fs + history_ic:
```

```
        w.writerow(row)
```

```
print(f'Saved {csv_path}')
```

```
# Print summary
```

```
print('\n--- Final epoch metrics ---')
```

```
for h in (history_fs[-1], history_ic[-1]):
```

```
    print(f"  {h['name']:14s}  loss={h['eval_loss']:.4f}  SSIM={h['eval_ssim']:.4f}  PSNR={h['eval_psnr']:.2f}dB
```

```
Saved /content/drive/MyDrive/FedSafeFuse/results/smoke_test.csv
```

```
--- Final epoch metrics ---
```

```
FedSafe-Fuse    loss=1.0734  SSIM=0.1819  PSNR=10.53dB  2.46s/epoch
```

```
IFCNN          loss=0.0262  SSIM=0.9835  PSNR=36.20dB  2.61s/epoch
```

```
# 6. Qualitative panel: 4 samples x [MRI, PET, FedSafe fused, IFCNN fused]
```

```
fs.eval(); ic.eval()
```

```
with torch.no_grad():
```

```
    mri, pet, _ = next(iter(eval_loader))
```

```
    mri_d, pet_d = mri.to(DEVICE), pet.to(DEVICE)
```

```
    fused_fs = fs(mri_d, pet_d).cpu()
```

```
    fused_ic = ic(mri_d, pet_d).cpu()
```

```
n_show = 4
```

```
fig, axes = plt.subplots(n_show, 4, figsize=(12, 3 * n_show))
```

```
col_titles = ['MRI (input)', 'PET (input)', 'FedSafe-Fuse', 'IFCNN baseline']
```

```
for i in range(n_show):
```

```
    panels = [mri[i, 0].numpy(), pet[i, 0].numpy(), fused_fs[i, 0].numpy(), fused_ic[i, 0].numpy()]
```

```
    for j, panel in enumerate(panels):
```

```
        axes[i, j].imshow(panel, cmap='gray', vmin=0, vmax=1)
```

```
        axes[i, j].axis('off')
```

```
        if i == 0:
```

```
            axes[i, j].set_title(col_titles[j])
```

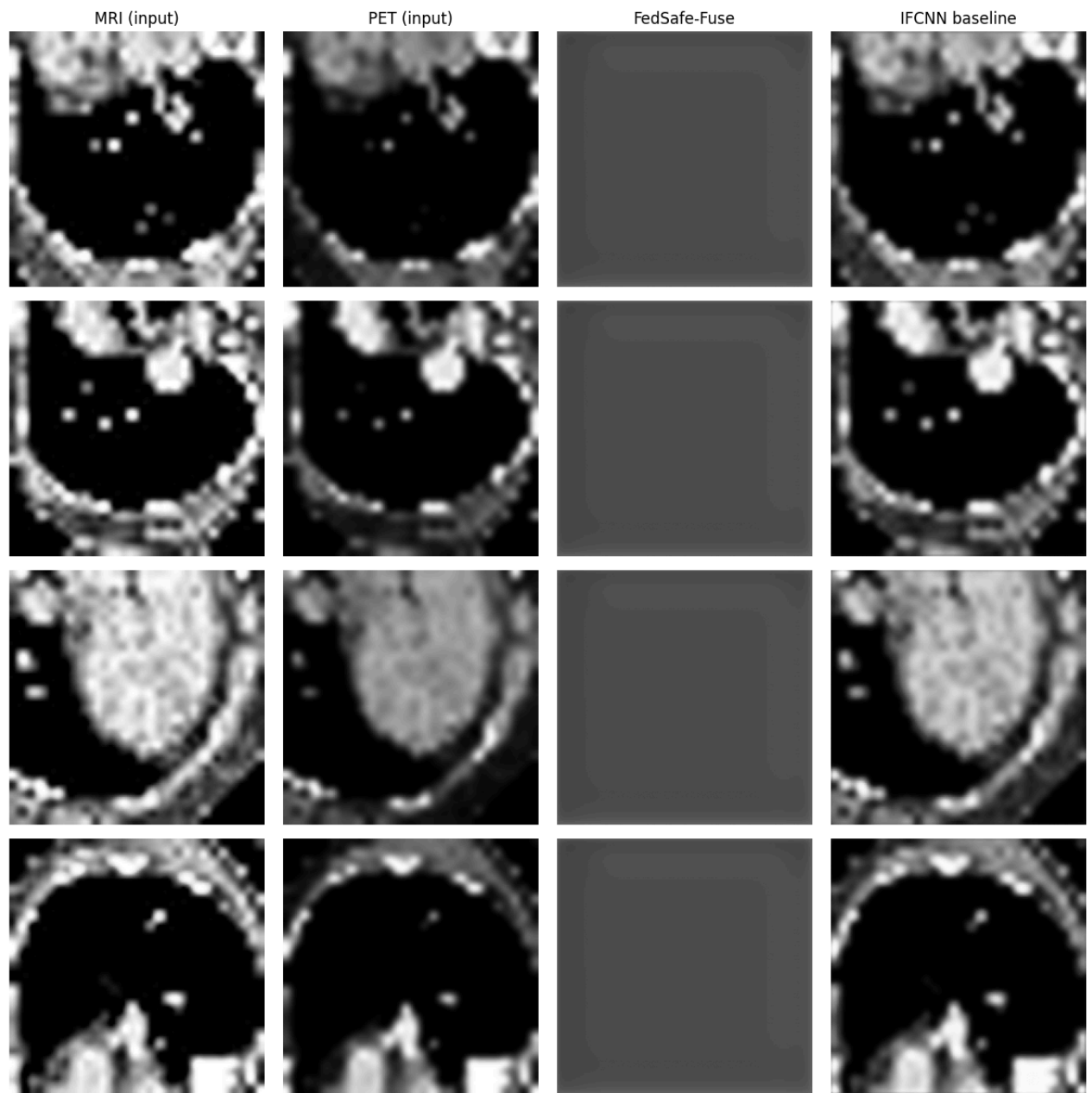
```
plt.tight_layout()
```

```
fig_path = f'{PROJECT_DRIVE}/figures/smoke_qualitative.png'
```

```
plt.savefig(fig_path, dpi=140, bbox_inches='tight')
```

```
plt.show()
```

```
print(f'Saved {fig_path}')
```

Saved /content/drive/MyDrive/FedSafeFuse/figures/smoke_qualitative.png

```
# 7. Timing extrapolation for the full federated schedule

# What we measured: seconds per epoch on a 256-sample subset (16 batches)
batches_per_epoch_smoke = history_fs[-1]['batches_per_epoch']
secs_per_batch_fs = history_fs[-1]['epoch_secs'] / batches_per_epoch_smoke
secs_per_batch_ic = history_ic[-1]['epoch_secs'] / batches_per_epoch_smoke

# Proposal config: K=3 clients, T=50 rounds, E=5 local epochs, batch=16
# Per-client data: Client 0 has ~7714 MedMNIST samples (from Phase 1 partition)
# Batches per local epoch (per client) ~= ceil(7714 / 16) ~= 483
PROPOSAL_K = 3
PROPOSAL_T = 50
PROPOSAL_E = 5

n_per_client = [len(c) for c in medmnist_partition['client_indices']]
print(f'Per-client sample counts: {n_per_client}')

# Estimate worst-case (largest client) batches per local epoch
batches_per_local_epoch = max(n_per_client) // 16 + 1
print(f'Worst-case batches per local epoch: {batches_per_local_epoch}')

# Sequential federated simulation: each round = K * E local epochs end-to-end
batches_per_round = PROPOSAL_K * PROPOSAL_E * batches_per_local_epoch
secs_per_round_fs = batches_per_round * secs_per_batch_fs
secs_per_round_ic = batches_per_round * secs_per_batch_ic
```

```
total_secs_fs = secs_per_round_fs * PROPOSAL_T
total_secs_ic = secs_per_round_ic * PROPOSAL_T

def fmt(s):
    return f'{s/60:.1f} min' if s < 7200 else f'{s/3600:.1f} h'

report = []
report.append(f'=== Timing extrapolation (MedMNIST, K={PROPOSAL_K}, T={PROPOSAL_T}, E={PROPOSAL_E}) ===')
report.append(f'Measured: FedSafe-Fuse {secs_per_batch_fs*1000:.0f} ms/batch | IFCNN {secs_per_batch_ic*1000:.0f} ms/batch')
report.append(f'Per round (K*E*batches): {batches_per_round} batches')
report.append(f' FedSafe-Fuse per round: {fmt(secs_per_round_fs)}')
report.append(f' IFCNN baseline per round: {fmt(secs_per_round_ic)}')
report.append(f'Full T={PROPOSAL_T} rounds:')
report.append(f' FedSafe-Fuse: ~{fmt(total_secs_fs)}')
report.append(f' IFCNN: ~{fmt(total_secs_ic)}')
report.append(f'Colab Free session limit: ~12h. Idle disconnect: ~90 min.')

# Verdict
budget_h = 12
if total_secs_fs / 3600 < budget_h * 0.8:
    report.append('VERDICT: Full T=50 fits comfortably in one Colab Free session.')
elif total_secs_fs / 3600 < budget_h:
    report.append('VERDICT: Full T=50 might fit in one session but is tight; checkpoint every 5 rounds.')
else:
    report.append('VERDICT: Full T=50 does NOT fit one session. Discuss reducing T or splitting across sessions.')

txt = '\n'.join(report)
print(txt)

txt_path = f'{PROJECT_DRIVE}/results/timing_extrapolation.txt'
with open(txt_path, 'w') as f:
    f.write(txt)
print(f'\nSaved {txt_path}')
```

Per-client sample counts: [7714, 15445, 6218]
Worst-case batches per local epoch: 966
=== Timing extrapolation (MedMNIST, K=3, T=50, E=5) ===
Measured: FedSafe-Fuse 154 ms/batch | IFCNN 163 ms/batch
Per round (K*E*batches): 14490 batches
FedSafe-Fuse per round: 37.1 min
IFCNN baseline per round: 39.4 min
Full T=50 rounds:
FedSafe-Fuse: ~30.9 h
IFCNN: ~32.9 h
Colab Free session limit: ~12h. Idle disconnect: ~90 min.
VERDICT: Full T=50 does NOT fit one session. Discuss reducing T or splitting across sessions.

Saved /content/drive/MyDrive/FedSafeFuse/results/timing_extrapolation.txt

When done

Paste back to chat:

- 1. The **parameter counts** (cell 2 output).
- 2. The **final-epoch metrics summary** (cell 5 output, the two lines).
- 3. The **timing extrapolation** block (cell 7 output, ~10 lines ending with `VERDICT:)`).
- 4. The qualitative panel `smoke_qualitative.png` (drag into chat).

Files to download back to your local repo:

Drive path	Local destination
results/smoke_test.csv	results/smoke_test.csv
results/timing_extrapolation.txt	results/timing_extrapolation.txt
figures/smoke_qualitative.png	figures/smoke_qualitative.png